

KRUTIK CYBER EXPERT

SQL COMPLETE COURSE

Queries | Joins | Functions | All Commands

BY KRUTIK 
CYBER EXPERT

"Learn Free. Manage Data Securely."

CHAPTER 1: SQL INTRODUCTION

What is SQL?

- Structured Query Language
- Created by IBM in 1970s
- Standard: ANSI SQL (Latest: SQL:2023)
- Used to manage relational databases

Why SQL?

- ✓ Most important skill for data
- ✓ Used in all web applications
- ✓ Works with all databases (MySQL, PostgreSQL, SQLite, Oracle, SQL Server)
- ✓ Easy to learn
- ✓ High demand in jobs

What can you do?

- ✓ Create databases and tables
- ✓ Insert, update, delete data
- ✓ Query and filter data
- ✓ Join multiple tables
- ✓ Aggregate data (sum, count, avg)

✓ Create views, indexes, stored procedures

SQL COMMANDS CATEGORIES:

DDL (Data Definition Language) - CREATE, ALTER, DROP, TRUNCATE

DML (Data Manipulation Language) - INSERT, UPDATE, DELETE

DQL (Data Query Language) - SELECT

DCL (Data Control Language) - GRANT, REVOKE

TCL (Transaction Control Language) - COMMIT, ROLLBACK, SAVEPOINT

CHAPTER 2: DATABASE AND TABLE CREATION (DDL)

A) CREATE DATABASE:

CODE:

-- Create database

CREATE DATABASE school_db;

-- Use database

USE school_db;

-- Drop database

DROP DATABASE school_db;

B) CREATE TABLE:

CODE:

-- Basic table

```
CREATE TABLE students (  
    id INT,  
    name VARCHAR(100),  
    age INT,  
    email VARCHAR(100)  
);
```

-- With constraints

```
CREATE TABLE students (
  id INT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  age INT CHECK (age >= 0),
  email VARCHAR(100) UNIQUE,
  city VARCHAR(50) DEFAULT 'Mumbai',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- With auto-increment

```
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100) NOT NULL,
  age INT,
  email VARCHAR(100) UNIQUE
);
```

C) DATA TYPES:

Type	Description
INT	Integer numbers
BIGINT	Large integer
DECIMAL(10,2)	Decimal number (10 digits, 2 dec)
VARCHAR(255)	Variable length string
CHAR(10)	Fixed length string
TEXT	Long text
DATE	Date (YYYY-MM-DD)
DATETIME	Date and time
TIMESTAMP	Timestamp
BOOLEAN	True/False
JSON	JSON data (MySQL 5.7+)

D) ALTER TABLE:

CODE:

-- Add column

```
ALTER TABLE students ADD COLUMN phone VARCHAR(15);
```

-- Drop column

ALTER TABLE students DROP COLUMN phone;

-- Modify column

ALTER TABLE students MODIFY COLUMN age INT NOT NULL;

-- Rename column

ALTER TABLE students RENAME COLUMN age TO student_age;

-- Rename table

ALTER TABLE students RENAME TO student_info;

-- Add constraint

ALTER TABLE students ADD CONSTRAINT pk_student PRIMARY KEY (id);

E) DROP TABLE:

CODE:

-- Delete table

DROP TABLE students;

-- Delete table if exists (no error)

DROP TABLE IF EXISTS students;

-- Delete all rows but keep table

TRUNCATE TABLE students;

CHAPTER 3: INSERT DATA (DML)

A) INSERT SINGLE ROW:

CODE:

-- Insert with all columns

INSERT INTO students (id, name, age, email, city)

VALUES (1, 'Krutik', 25, 'krutik@example.com', 'Mumbai');

-- Insert without specifying all columns (needs default/NULL)

INSERT INTO students (name, age, email)

VALUES ('Rahul', 22, 'rahul@example.com');

-- Insert with auto-increment

```
INSERT INTO students (name, age, email)
VALUES ('Priya', 23, 'priya@example.com');
```

-- Insert with default value

```
INSERT INTO students (name, age, email, city)
VALUES ('Amit', 24, 'amit@example.com', DEFAULT);
```

B) INSERT MULTIPLE ROWS:

CODE:

```
INSERT INTO students (name, age, email, city) VALUES
('Krutik', 25, 'krutik@example.com', 'Mumbai'),
('Rahul', 22, 'rahul@example.com', 'Delhi'),
('Priya', 23, 'priya@example.com', 'Bangalore'),
('Amit', 24, 'amit@example.com', 'Mumbai'),
('Sneha', 21, 'sneha@example.com', 'Pune');
```

C) INSERT FROM ANOTHER TABLE:

CODE:

```
INSERT INTO students_archive (name, age, email)
SELECT name, age, email FROM students WHERE age > 25;
```

CHAPTER 4: SELECT (Query Data - DQL)

A) BASIC SELECT:

CODE:

-- Select all columns
SELECT * FROM students;

-- Select specific columns
SELECT name, age, email FROM students;

-- Select with alias (renaming)

```
SELECT name AS student_name, age AS student_age FROM students;
```

B) SELECT WITH WHERE (Filtering):

CODE:

-- Equal

```
SELECT * FROM students WHERE age = 25;
```

-- Not equal

```
SELECT * FROM students WHERE age != 25;
```

```
SELECT * FROM students WHERE age <> 25;
```

-- Greater/Less than

```
SELECT * FROM students WHERE age > 22;
```

```
SELECT * FROM students WHERE age >= 22;
```

```
SELECT * FROM students WHERE age < 22;
```

```
SELECT * FROM students WHERE age <= 22;
```

-- Between

```
SELECT * FROM students WHERE age BETWEEN 20 AND 25;
```

-- In (list)

```
SELECT * FROM students WHERE city IN ('Mumbai', 'Delhi');
```

```
SELECT * FROM students WHERE city NOT IN ('Mumbai', 'Delhi');
```

-- Like (pattern matching)

```
SELECT * FROM students WHERE name LIKE 'K%'; -- Starts with K
```

```
SELECT * FROM students WHERE name LIKE '%ul'; -- Ends with ul
```

```
SELECT * FROM students WHERE name LIKE '%it%'; -- Contains it
```

```
SELECT * FROM students WHERE name LIKE 'A__t'; -- A followed by 2 chars, then t
```

-- NULL checks

```
SELECT * FROM students WHERE phone IS NULL;
```

```
SELECT * FROM students WHERE phone IS NOT NULL;
```

C) AND / OR / NOT:

CODE:

-- AND (both true)

```
SELECT * FROM students WHERE age > 22 AND city = 'Mumbai';
```

-- OR (at least one true)

SELECT * FROM students WHERE city = 'Mumbai' OR city = 'Delhi';

-- NOT (reverse condition)

SELECT * FROM students WHERE NOT city = 'Mumbai';

D) ORDER BY (Sorting):

CODE:

-- Ascending (default)

SELECT * FROM students ORDER BY age;

-- Descending

SELECT * FROM students ORDER BY age DESC;

-- Multiple columns

SELECT * FROM students ORDER BY city ASC, age DESC;

E) DISTINCT (Unique values):

CODE:

-- Get unique cities

SELECT DISTINCT city FROM students;

-- Unique combination

SELECT DISTINCT city, age FROM students;

F) LIMIT (Restrict rows):

CODE:

-- First 5 rows

SELECT * FROM students LIMIT 5;

-- Skip 2, get 5 rows (pagination)

SELECT * FROM students LIMIT 5 OFFSET 2;

SELECT * FROM students LIMIT 2, 5; -- MySQL syntax

G) OFFSET FETCH (SQL Server/PostgreSQL):

CODE:

-- Skip 2, get 5

SELECT * FROM students OFFSET 2 ROWS FETCH NEXT 5 ROWS ONLY;

CHAPTER 5: UPDATE AND DELETE

A) UPDATE:

CODE:

-- Update single column

UPDATE students SET age = 26 WHERE name = 'Krutik';

-- Update multiple columns

UPDATE students SET age = 26, city = 'Mumbai' WHERE name = 'Krutik';

-- Update all rows (be careful!)

UPDATE students SET age = 30;

-- Use WHERE to avoid this

-- Update with calculations

UPDATE students SET age = age + 1 WHERE city = 'Mumbai';

B) DELETE:

CODE:

-- Delete specific rows

DELETE FROM students WHERE name = 'Krutik';

-- Delete all rows (be careful!)

DELETE FROM students;

-- Delete with condition

DELETE FROM students WHERE age < 18;

C) DELETE vs TRUNCATE:

DELETE	TRUNCATE
Removes rows	Removes all rows
Can use WHERE	No WHERE allowed
Slower	Faster
Resets auto-increment	Resets auto-increment
Can be rolled back	Can be rolled back in transaction
Triggers fire	Triggers don't fire

CHAPTER 6: AGGREGATE FUNCTIONS

A) COMMON AGGREGATE FUNCTIONS:

CODE:

```
-- COUNT - Count rows
SELECT COUNT(*) FROM students;    -- Total students
SELECT COUNT(email) FROM students; -- Count non-null emails
SELECT COUNT(DISTINCT city) FROM students; -- Count unique cities
```

```
-- SUM - Sum of values
SELECT SUM(age) FROM students;    -- Total of all ages
```

```
-- AVG - Average
SELECT AVG(age) FROM students;    -- Average age
```

```
-- MIN - Minimum value
SELECT MIN(age) FROM students;    -- Youngest age
```

```
-- MAX - Maximum value
SELECT MAX(age) FROM students;    -- Oldest age
```

B) GROUP BY (Group rows):

CODE:

```
-- Group by city
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city;
```

```
-- Group with aggregate
SELECT city, AVG(age) AS avg_age
FROM students
GROUP BY city;
```

```
-- Multiple columns
SELECT city, age, COUNT(*) AS count
FROM students
GROUP BY city, age;
```

C) HAVING (Filter groups):

CODE:

```
-- HAVING is used after GROUP BY (WHERE is before)
SELECT city, COUNT(*) AS count
FROM students
GROUP BY city
HAVING COUNT(*) > 2;
```

```
-- HAVING with aggregate
SELECT city, AVG(age) AS avg_age
FROM students
GROUP BY city
HAVING AVG(age) > 23;
```

D) ROLLUP / CUBE (MySQL 8+):

CODE:

```
-- ROLLUP - Adds subtotal rows
SELECT city, gender, COUNT(*) AS count
FROM students
GROUP BY city, gender WITH ROLLUP;
```

CHAPTER 7: JOINS (Combining Tables)

A) SAMPLE TABLES:

CODE:

```
-- Students table
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(100),
    class_id INT
);

-- Classes table
CREATE TABLE classes (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    teacher VARCHAR(50)
);

-- Sample data
INSERT INTO students VALUES (1, 'Krutik', 1);
INSERT INTO students VALUES (2, 'Rahul', 1);
INSERT INTO students VALUES (3, 'Priya', 2);
INSERT INTO students VALUES (4, 'Amit', NULL);

INSERT INTO classes VALUES (1, 'Mathematics', 'Mr. Sharma');
INSERT INTO classes VALUES (2, 'Science', 'Ms. Patel');
INSERT INTO classes VALUES (3, 'English', 'Mrs. Singh');
```

B) INNER JOIN (Only matching rows):

CODE:

```
-- INNER JOIN - returns matching rows
SELECT students.name, classes.name AS class, classes.teacher
FROM students
INNER JOIN classes ON students.class_id = classes.id;

-- Or simpler syntax
```

```
SELECT s.name, c.name AS class, c.teacher
FROM students s
JOIN classes c ON s.class_id = c.id;
```

C) LEFT JOIN (All from left + matching from right):

CODE:

```
-- LEFT JOIN - all students, even without class
SELECT s.name, c.name AS class, c.teacher
FROM students s
LEFT JOIN classes c ON s.class_id = c.id;
```

D) RIGHT JOIN (All from right + matching from left):

CODE:

```
-- RIGHT JOIN - all classes, even without students
SELECT s.name, c.name AS class, c.teacher
FROM students s
RIGHT JOIN classes c ON s.class_id = c.id;
```

E) FULL OUTER JOIN (All rows from both):

CODE:

```
-- FULL OUTER JOIN - all students and all classes
-- MySQL doesn't support FULL OUTER JOIN directly
```

```
SELECT s.name, c.name AS class, c.teacher
FROM students s
LEFT JOIN classes c ON s.class_id = c.id
```

UNION

```
SELECT s.name, c.name AS class, c.teacher
FROM students s
RIGHT JOIN classes c ON s.class_id = c.id;
```

F) CROSS JOIN (Cartesian product):

CODE:

```
-- CROSS JOIN - every student with every class
SELECT s.name, c.name AS class
FROM students s
CROSS JOIN classes c;
```

G) SELF JOIN (Join table with itself):

CODE:

```
-- Find employees and their managers
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  manager_id INT
);
```

```
SELECT e.name AS employee, m.name AS manager
FROM employees e
LEFT JOIN employees m ON e.manager_id = m.id;
```

H) MULTIPLE JOINS:

CODE:

```
SELECT s.name, c.name AS class, t.name AS teacher
FROM students s
JOIN classes c ON s.class_id = c.id
JOIN teachers t ON c.teacher_id = t.id;
```

CHAPTER 8: SUBQUERIES

A) SUBQUERY IN WHERE:

CODE:

```
-- Students older than average
SELECT * FROM students
WHERE age > (SELECT AVG(age) FROM students);
```

```
-- Students in class with most students
SELECT * FROM students
WHERE class_id = (
    SELECT class_id
    FROM students
    GROUP BY class_id
    ORDER BY COUNT(*) DESC
    LIMIT 1
);
```

B) SUBQUERY IN SELECT:

CODE:

```
-- With subquery in SELECT
SELECT
    name,
    age,
    (SELECT AVG(age) FROM students) AS avg_age,
    age - (SELECT AVG(age) FROM students) AS diff
FROM students;
```

C) EXISTS / NOT EXISTS:

CODE:

```
-- Students who have any class
SELECT * FROM students s
WHERE EXISTS (
    SELECT 1 FROM classes c WHERE c.id = s.class_id
);
```

```
-- Students with no class
SELECT * FROM students s
WHERE NOT EXISTS (
    SELECT 1 FROM classes c WHERE c.id = s.class_id
);
```

D) ANY / ALL:

CODE:

-- Age greater than any student in Mumbai

SELECT * FROM students

WHERE age > ANY (

SELECT age FROM students WHERE city = 'Mumbai'

);

-- Age greater than all students in Mumbai

SELECT * FROM students

WHERE age > ALL (

SELECT age FROM students WHERE city = 'Mumbai'

);

CHAPTER 9: SET OPERATIONS

A) UNION (Combine results):

CODE:

-- UNION - removes duplicates

SELECT name FROM students WHERE city = 'Mumbai'

UNION

SELECT name FROM students WHERE age > 22;

-- UNION ALL - keeps duplicates

SELECT name FROM students WHERE city = 'Mumbai'

UNION ALL

SELECT name FROM students WHERE age > 22;

B) INTERSECT (Common rows):

CODE:

-- MySQL doesn't support INTERSECT directly

SELECT name FROM students WHERE city = 'Mumbai'

INTERSECT

SELECT name FROM students WHERE age > 22;

```
-- Alternative using IN
SELECT name FROM students
WHERE city = 'Mumbai'
AND name IN (SELECT name FROM students WHERE age > 22);
```

C) EXCEPT (Rows in first but not second):

CODE:

```
-- MySQL doesn't support EXCEPT directly
SELECT name FROM students WHERE city = 'Mumbai'
EXCEPT
SELECT name FROM students WHERE age > 22;
```

```
-- Alternative using NOT IN
SELECT name FROM students
WHERE city = 'Mumbai'
AND name NOT IN (SELECT name FROM students WHERE age > 22);
```

CHAPTER 10: INDEXES AND CONSTRAINTS

A) CONSTRAINTS:

CODE:

```
-- PRIMARY KEY - Unique identifier
CREATE TABLE students (
    id INT PRIMARY KEY
);
```

```
-- FOREIGN KEY - References another table
CREATE TABLE orders (
    id INT PRIMARY KEY,
    student_id INT,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

```
-- UNIQUE - No duplicates allowed
CREATE TABLE users (
```



```
email VARCHAR(100) UNIQUE  
);
```

```
-- NOT NULL - Cannot be empty  
CREATE TABLE students (  
    name VARCHAR(100) NOT NULL  
);
```

```
-- CHECK - Validate data  
CREATE TABLE students (  
    age INT CHECK (age >= 0 AND age <= 150)  
);
```

```
-- DEFAULT - Default value  
CREATE TABLE students (  
    city VARCHAR(50) DEFAULT 'Mumbai'  
);
```

B) INDEXES (Speed up queries):

CODE:

```
-- Create index  
CREATE INDEX idx_student_name ON students(name);  
  
-- Create unique index  
CREATE UNIQUE INDEX idx_student_email ON students(email);  
  
-- Create composite index (multiple columns)  
CREATE INDEX idx_student_city_age ON students(city, age);  
  
-- Drop index  
DROP INDEX idx_student_name ON students;  
  
-- View indexes  
SHOW INDEX FROM students;
```

C) WHEN TO USE INDEXES:

-  Use when:
- Large tables (> 10,000 rows)
 - Columns used in WHERE, JOIN, ORDER BY
 - Foreign key columns
 - Columns with high cardinality (many unique values)

 Avoid when:

- Small tables
- Frequently updated columns
- Columns with few unique values (e.g., gender)

CHAPTER 11: VIEWS

A) CREATE VIEW:

CODE:

```
-- Create view (virtual table)
CREATE VIEW student_details AS
SELECT s.id, s.name, s.age, c.name AS class_name, c.teacher
FROM students s
LEFT JOIN classes c ON s.class_id = c.id;
```

```
-- Use view like table
SELECT * FROM student_details WHERE age > 22;
```

B) CREATE VIEW WITH CHECK OPTION:

CODE:

```
-- With check option (prevents invalid updates)
CREATE VIEW mumbai_students AS
SELECT * FROM students WHERE city = 'Mumbai'
WITH CHECK OPTION;
```

C) UPDATE/DELETE VIEW:

CODE:

```
-- Update view (updates underlying table)
UPDATE student_details SET age = 26 WHERE name = 'Krutik';
```

```
-- Drop view
DROP VIEW student_details;
```

CHAPTER 12: STORED PROCEDURES

A) CREATE PROCEDURE:

CODE:

```
-- Simple procedure
DELIMITER //
CREATE PROCEDURE GetStudents()
BEGIN
    SELECT * FROM students;
END //
DELIMITER ;

-- Call procedure
CALL GetStudents();
```

B) PROCEDURE WITH PARAMETERS:

CODE:

```
DELIMITER //
CREATE PROCEDURE GetStudentsByCity(IN city_name VARCHAR(50))
BEGIN
    SELECT * FROM students WHERE city = city_name;
END //
DELIMITER ;

CALL GetStudentsByCity('Mumbai');
```

C) PROCEDURE WITH OUTPUT:

CODE:

```
DELIMITER //
CREATE PROCEDURE GetStudentCount(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total FROM students;
```

END //
DELIMITER ;

CALL GetStudentCount(@total);
SELECT @total;

D) DROP PROCEDURE:

CODE:

DROP PROCEDURE GetStudents;

CHAPTER 13: TRANSACTIONS (TCL)

A) BASIC TRANSACTION:

CODE:

START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;

-- If everything is fine
COMMIT;

-- If something went wrong
ROLLBACK;

B) SAVEPOINT:

CODE:

START TRANSACTION;

UPDATE accounts SET balance = balance - 100 WHERE id = 1;
SAVEPOINT sp1;

```
UPDATE accounts SET balance = balance + 100 WHERE id = 2;  
-- Oops, wrong account
```

```
ROLLBACK TO SAVEPOINT sp1; -- Undo last update
```

```
COMMIT; -- Only first update is applied
```

C) AUTOCOMMIT:

CODE:

```
-- Turn off autocommit (MySQL)  
SET AUTOCOMMIT = 0;
```

```
-- Turn on  
SET AUTOCOMMIT = 1;
```

```
-- Check status  
SELECT @@AUTOCOMMIT;
```

CHAPTER 14: STRING FUNCTIONS

A) COMMON STRING FUNCTIONS:

CODE:

```
-- CONCAT - Combine strings  
SELECT CONCAT(name, ' - ', city) AS info FROM students;
```

```
-- CONCAT_WS - With separator  
SELECT CONCAT_WS(' | ', name, age, city) AS info FROM students;
```

```
-- LENGTH - Length of string  
SELECT name, LENGTH(name) AS name_length FROM students;
```

```
-- UPPER / LOWER  
SELECT UPPER(name), LOWER(name) FROM students;
```

```
-- SUBSTRING / SUBSTR  
SELECT SUBSTRING(name, 1, 3) FROM students; -- First 3 chars
```

-- LEFT / RIGHT

SELECT LEFT(name, 2), RIGHT(name, 2) FROM students;

-- TRIM / LTRIM / RTRIM

SELECT TRIM(' Hello ') AS trimmed; -- 'Hello'

SELECT LTRIM(' Hello ') AS left_trimmed; -- 'Hello '

SELECT RTRIM(' Hello ') AS right_trimmed; -- ' Hello'

-- REPLACE

SELECT REPLACE(name, 'a', 'x') FROM students;

-- POSITION / LOCATE (find substring)

SELECT POSITION('it' IN name) FROM students; -- Position of 'it'

-- REVERSE

SELECT REVERSE(name) FROM students;

-- FORMAT (format numbers)

SELECT FORMAT(1234.567, 2); -- '1,234.57'

CHAPTER 15: DATE AND TIME FUNCTIONS

A) CURRENT DATE/TIME:

CODE:

-- Current date and time

SELECT NOW(); -- 2026-08-10 14:30:00

SELECT CURDATE(); -- 2026-08-10

SELECT CURTIME(); -- 14:30:00

SELECT CURRENT_DATE(); -- 2026-08-10

SELECT CURRENT_TIME(); -- 14:30:00

SELECT CURRENT_TIMESTAMP(); -- 2026-08-10 14:30:00

B) EXTRACT DATE PARTS:

CODE:

SELECT YEAR(NOW()); -- 2026

```

SELECT MONTH(NOW());      -- 8
SELECT DAY(NOW());        -- 10
SELECT HOUR(NOW());       -- 14
SELECT MINUTE(NOW());     -- 30
SELECT SECOND(NOW());     -- 0
SELECT DAYOFWEEK(NOW());  -- 2 (Monday)
SELECT DAYOFMONTH(NOW()); -- 10
SELECT DAYOFYEAR(NOW());  -- 222
SELECT WEEK(NOW());       -- 32
SELECT QUARTER(NOW());    -- 3

```

C) DATE FORMATTING:

CODE:

```

-- DATE_FORMAT (MySQL)
SELECT DATE_FORMAT(NOW(), '%Y-%m-%d'); -- 2026-08-10
SELECT DATE_FORMAT(NOW(), '%d/%m/%Y'); -- 10/08/2026
SELECT DATE_FORMAT(NOW(), '%W, %M %d %Y'); -- Monday, August 10 2026

```

```

-- DATE_ADD / DATE_SUB
SELECT DATE_ADD(NOW(), INTERVAL 7 DAY); -- +7 days
SELECT DATE_SUB(NOW(), INTERVAL 1 MONTH); -- -1 month
SELECT NOW() + INTERVAL 7 DAY;          -- +7 days

```

```

-- DATEDIFF - Difference in days
SELECT DATEDIFF('2026-12-31', '2026-01-01'); -- 364

```

D) DATE CONVERSION:

CODE:

```

-- STR_TO_DATE - String to date
SELECT STR_TO_DATE('10-08-2026', '%d-%m-%Y');

```

```

-- CAST - Convert to date
SELECT CAST('2026-08-10' AS DATE);

```

CHAPTER 16: ADVANCED QUERIES

A) WINDOW FUNCTIONS (MySQL 8+):

CODE:

-- ROW_NUMBER - Sequential number

```
SELECT
    name,
    age,
    ROW_NUMBER() OVER (ORDER BY age DESC) AS rank_no
FROM students;
```

-- RANK - With ties

```
SELECT
    name,
    age,
    RANK() OVER (ORDER BY age DESC) AS rank
FROM students;
```

-- DENSE_RANK - Without gaps

```
SELECT
    name,
    age,
    DENSE_RANK() OVER (ORDER BY age DESC) AS dense_rank
FROM students;
```

-- PARTITION BY - Group-wise rank

```
SELECT
    city,
    name,
    age,
    RANK() OVER (PARTITION BY city ORDER BY age DESC) AS rank_in_city
FROM students;
```

-- LAG / LEAD - Previous/Next value

```
SELECT
    name,
    age,
    LAG(age) OVER (ORDER BY age) AS prev_age,
    LEAD(age) OVER (ORDER BY age) AS next_age
FROM students;
```

B) CTE (Common Table Expression - WITH clause):

CODE:

```

-- Simple CTE
WITH student_ages AS (
    SELECT name, age FROM students WHERE age > 22
)
SELECT * FROM student_ages;

-- Recursive CTE (Hierarchical data)
WITH RECURSIVE employee_hierarchy AS (
    -- Base case
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case
    SELECT e.id, e.name, e.manager_id, eh.level + 1
    FROM employees e
    INNER JOIN employee_hierarchy eh ON e.manager_id = eh.id
)
SELECT * FROM employee_hierarchy;

```

C) CASE STATEMENT:

CODE:

```

-- Simple CASE
SELECT
    name,
    age,
    CASE age
        WHEN 21 THEN 'Twenty One'
        WHEN 22 THEN 'Twenty Two'
        ELSE 'Other'
    END AS age_group
FROM students;

-- Searched CASE
SELECT
    name,
    age,
    CASE
        WHEN age < 18 THEN 'Minor'
        WHEN age BETWEEN 18 AND 24 THEN 'Young Adult'
        WHEN age BETWEEN 25 AND 60 THEN 'Adult'
    END AS age_group
FROM students;

```

```
        ELSE 'Senior'
    END AS age_category
FROM students;
```

D) COALESCE and NULLIF:

CODE:

```
-- COALESCE - First non-null value
SELECT name, COALESCE(phone, 'No Phone') AS phone FROM students;
```

```
-- NULLIF - Returns NULL if equal
SELECT NULLIF(age, 25) AS age FROM students; -- NULL if age=25
```

CHAPTER 17: COMPLETE PROJECT - SCHOOL MANAGEMENT SYSTEM

SQL SCHEMA:

```
-- 1. Students table
CREATE TABLE students (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    date_of_birth DATE,
    gender ENUM('M', 'F', 'O'),
    address TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```
-- 2. Teachers table
CREATE TABLE teachers (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    phone VARCHAR(15),
    hire_date DATE,
    salary DECIMAL(10,2),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

-- 3. Courses table

```
CREATE TABLE courses (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  name VARCHAR(100) NOT NULL,  
  code VARCHAR(20) UNIQUE NOT NULL,  
  credits INT DEFAULT 3,  
  teacher_id INT,  
  FOREIGN KEY (teacher_id) REFERENCES teachers(id) ON DELETE SET NULL  
);
```

-- 4. Enrollments table (Many-to-Many)

```
CREATE TABLE enrollments (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  student_id INT NOT NULL,  
  course_id INT NOT NULL,  
  enrollment_date DATE DEFAULT CURDATE(),  
  grade CHAR(2),  
  FOREIGN KEY (student_id) REFERENCES students(id) ON DELETE CASCADE,  
  FOREIGN KEY (course_id) REFERENCES courses(id) ON DELETE CASCADE,  
  UNIQUE KEY unique_enrollment (student_id, course_id)  
);
```

-- 5. Insert sample data

```
INSERT INTO students (name, email, phone, date_of_birth, gender) VALUES  
('Krutik Sharma', 'krutik@example.com', '9876543210', '2000-01-15', 'M'),  
('Rahul Verma', 'rahul@example.com', '9876543211', '2001-06-20', 'M'),  
('Priya Patel', 'priya@example.com', '9876543212', '2000-12-01', 'F'),  
('Amit Kumar', 'amit@example.com', '9876543213', '2002-03-10', 'M');
```

```
INSERT INTO teachers (name, email, phone, hire_date, salary) VALUES  
('Mr. Sharma', 'sharma@example.com', '9876543220', '2018-07-01', 50000),  
('Ms. Patel', 'patel@example.com', '9876543221', '2019-08-15', 45000),  
('Mrs. Singh', 'singh@example.com', '9876543222', '2020-01-10', 48000);
```

```
INSERT INTO courses (name, code, credits, teacher_id) VALUES  
('Mathematics', 'MATH101', 4, 1),  
('Physics', 'PHY101', 3, 2),  
('Computer Science', 'CS101', 4, 3),  
('Chemistry', 'CHE101', 3, 2),  
('English', 'ENG101', 2, 3);
```

```
INSERT INTO enrollments (student_id, course_id, enrollment_date, grade) VALUES  
(1, 1, '2026-01-15', 'A'),  
(1, 2, '2026-01-15', 'B+'),  
(1, 3, '2026-01-15', 'A-'),  
(2, 1, '2026-01-15', 'B'),  
(2, 4, '2026-01-15', 'A'),
```

```
(3, 2, '2026-01-15', 'A+'),  
(3, 5, '2026-01-15', 'B+');
```

-- 6. Useful Queries

-- All students with their courses

```
SELECT  
    s.name AS student,  
    c.name AS course,  
    c.code AS course_code,  
    t.name AS teacher,  
    e.grade  
FROM students s  
JOIN enrollments e ON s.id = e.student_id  
JOIN courses c ON e.course_id = c.id  
JOIN teachers t ON c.teacher_id = t.id  
ORDER BY s.name;
```

-- Students with number of courses

```
SELECT  
    s.name,  
    COUNT(e.course_id) AS course_count  
FROM students s  
LEFT JOIN enrollments e ON s.id = e.student_id  
GROUP BY s.id  
ORDER BY course_count DESC;
```

-- Teachers with course load

```
SELECT  
    t.name AS teacher,  
    COUNT(c.id) AS course_count  
FROM teachers t  
LEFT JOIN courses c ON t.id = c.teacher_id  
GROUP BY t.id;
```

-- Top performing students (by grade)

```
SELECT  
    s.name,  
    AVG(CASE e.grade  
        WHEN 'A+' THEN 10  
        WHEN 'A' THEN 9  
        WHEN 'A-' THEN 8.5  
        WHEN 'B+' THEN 8  
        WHEN 'B' THEN 7  
        WHEN 'B-' THEN 6.5  
        WHEN 'C+' THEN 6  
        WHEN 'C' THEN 5  
        ELSE 0
```

```
    END) AS avg_score
FROM students s
JOIN enrollments e ON s.id = e.student_id
GROUP BY s.id
ORDER BY avg_score DESC;
```

-- View: Student transcript

```
CREATE VIEW student_transcript AS
SELECT
```

```
    s.id AS student_id,
    s.name AS student_name,
    c.name AS course_name,
    c.code AS course_code,
    t.name AS teacher_name,
    e.grade,
    e.enrollment_date
```

```
FROM students s
```

```
JOIN enrollments e ON s.id = e.student_id
```

```
JOIN courses c ON e.course_id = c.id
```

```
JOIN teachers t ON c.teacher_id = t.id;
```

-- View: Course enrollment summary

```
CREATE VIEW course_summary AS
SELECT
```

```
    c.id AS course_id,
    c.name AS course_name,
    t.name AS teacher,
    COUNT(e.student_id) AS student_count,
    COUNT(e.grade) AS graded_count
```

```
FROM courses c
```

```
LEFT JOIN teachers t ON c.teacher_id = t.id
```

```
LEFT JOIN enrollments e ON c.id = e.course_id
```

```
GROUP BY c.id;
```

CHAPTER 18: QUICK REFERENCE - ALL COMMANDS

DDL (Data Definition Language):

**CREATE DATABASE, DROP DATABASE, CREATE TABLE, ALTER TABLE
DROP TABLE, TRUNCATE TABLE, CREATE INDEX, DROP INDEX**

DML (Data Manipulation Language):

INSERT, UPDATE, DELETE

DQL (Data Query Language):

**SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, OFFSET
JOIN (INNER, LEFT, RIGHT, FULL, CROSS, SELF)**

TCL (Transaction Control):

START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT

DCL (Data Control):

GRANT, REVOKE

CONSTRAINTS:

PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT

AGGREGATE FUNCTIONS:

COUNT, SUM, AVG, MIN, MAX

STRING FUNCTIONS:

CONCAT, LENGTH, UPPER, LOWER, SUBSTRING, TRIM, REPLACE

DATE FUNCTIONS:

NOW, CURDATE, CURTIME, YEAR, MONTH, DAY, DATE_ADD, DATEDIFF

OTHER:

CASE, COALESCE, NULLIF, UNION, INTERSECT, EXCEPT

WINDOW FUNCTIONS (MySQL 8+):

ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, PARTITION BY

CERTIFICATE OF COMPLETION

This certifies that the reader has completed

SQL COMPLETE COURSE

By

KRUTIK  CYBER EXPERT

Topics Covered:

- ✓ Database & Table Creation (DDL)
- ✓ Data Types & Constraints
- ✓ Insert Data (INSERT)
- ✓ Query Data (SELECT, WHERE, ORDER BY)
- ✓ Update and Delete (UPDATE, DELETE)
- ✓ Aggregate Functions (COUNT, SUM, AVG, MIN, MAX)
- ✓ GROUP BY & HAVING

- ✓ Joins (INNER, LEFT, RIGHT, FULL, CROSS, SELF)
- ✓ Subqueries (WHERE, SELECT, EXISTS)
- ✓ Set Operations (UNION, INTERSECT, EXCEPT)
- ✓ Indexes & Constraints
- ✓ Views
- ✓ Stored Procedures
- ✓ Transactions (COMMIT, ROLLBACK)
- ✓ String Functions
- ✓ Date Functions
- ✓ Advanced Queries (Window Functions, CTE, CASE)
- ✓ Complete Project (School Management System)
- ✓ Quick Reference (All Commands)

"Master Data Management."

**NOTE: This PDF is completely FREE. If you paid for it, you got scammed.
Share it with everyone who wants to learn SQL! 🚀**
